

Relatório Técnico

Dados do Aluno

- Nome: *Lauro Camilo Barbosa Marques da Rocha*
- Link para o repositório do projeto: https://github.com/LMRocha/sistemas-coginitivos-llm_extrator-texto-rpg-regras

Disciplina

Sistemas Cognitivos com LLMs

Extrator de regras para jogos de Role Playing Game

Objetivo

Implementação de uma solução especialista em um determinado sistema de RPG (determinado na base de conhecimento) em recuperar regras, resumir cenários e gerar exemplos conforme a solicitação do usuário via prompt

Descrição resumida do problema e do fluxo

Dado uma base de conhecimento pré estabelecida, o usuário solicita uma determinada tarefa (resumo de regra, geração de exemplo..etc). A LLM carregada utilizando a base vetorial construída em cima dos documentos disponibilizados retorna a melhor resposta ao usuário

Corpus

Foi utilizado um documento PDF referente a um livro de regras básicas do sistema **Basic Fantasy Role-Playing Game** distribuído gratuitamente pela <https://www.basicfantasy.org/>.

Motivação do uso da LLM e o modelo utilizado

A opção pela LLM foi devido a facilidade da implementação e a necessidade de um modelo pré treinado para realizar a geração do texto solicitado pelo usuário com a maior quantidade de precisão.

Ferramentas e APIs

Não foram utilizadas ferramentas e APIs externas, sendo o modelo executado inteiramente local.

Motivação do modelo

O modelo escolhido foi o `Qwen/Qwen2.5-1.5B-Instruct` devido a sua boa performance para execução local e ao baixo requerimento em recursos, pois o terminal no qual foi desenvolvido tinha uma configuração de 16gb de RAM e uma GPU de 2gb (GTX 1050 TI).

Implementações NLP

Foi implementada a vetorização e a tokenização tanto do corpus como também do prompt de entrada.

Estratégias de prompting

O prompt base utilizou a estratégia de `few-shot prompting` baseado em contexto e `meta-prompting` na realização da checagem da resposta da LLM após sua execução

Prompts utilizados

Foi utilizado o seguinte prompt base:

```
```markdown
#ROLE
You are an role playing game expert.

Always prioritize the retrieved context when answering.

If the answer cannot be found in the retrieved context,
use your own knowledge but clearly indicate that the information
was not found in the documents.

Be concise, accurate and well-structured.

#CONTEXT

The following text was retrieved from a document database.

{{Context}}

#TASK

Answer the user's request, using the retrieved context as the primary source.

The user's request can be a question or a request like 'Explaing this...', 'Resume this..' fo

If the context contains only part of the answer:
- Complete the answer using your own knowledge.
- Clearly indicate which information comes from your own knowledge.

If the context does not contain the answer:
- State that the information was not found in the retrieved documents.
- Then answer using your general knowledge.

#USER REQUEST

{{question}}

#OUTPUT

Produce:
1. A complete answer.
2. A well-structured explanation.
3. Bullet points or numbered lists when appropriate.
4. Markdown formatting.

#CHECKLIST

Before answering, verify:
- The retrieved context was analyzed.
- Information from multiple retrieved passages was combined when relevant.
- No facts were invented as if they came from the documents.
```

```
❑❑The answer is complete and detailed.
❑❑The response directly answers the user's question.
\ \ \ \
```

Estratégia de avaliação dos prompts

Foi utilizada a estratégia de *Meta Prompting* para realizar um checklist antes da resposta ser exibida ao usuário. E prompts estruturados em *papel, contexto, tarefa e formato de saída*

Saidas estruturadas

Foi soliticada uma saida estruturada na seção # OUTPUT

Modelos utilizados para Embeddings

Foi utilizado o modelo `sentence-transformers/all-MiniLM-L6-v2`

Estratégia de busca vetorial

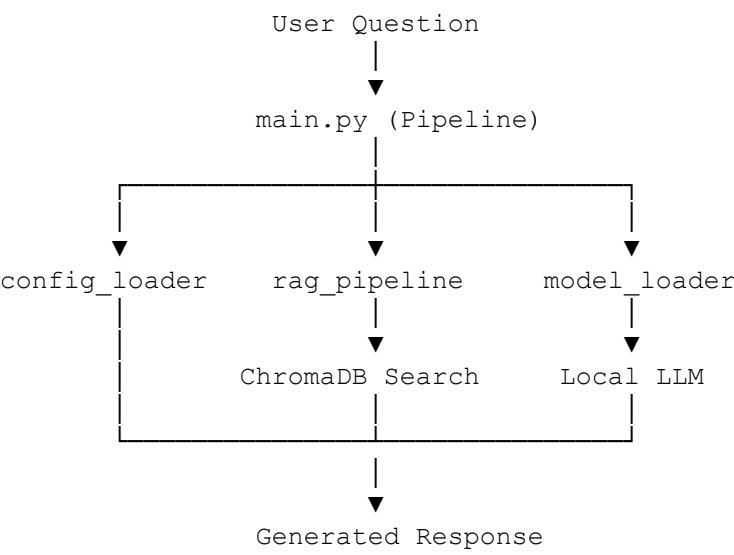
O retrieval realizado no chromedb é feito por `Busca por Similaridade`

Estratégia de execução

A execução do modelo foi realizada localmente através da biblioteca `Transformers` utilizando o modelo `Qwen/Qwen2.5-1.5B-Instruct`. A escolha de uso do modelo local não remete a privacidade dos dados utilizados até por que os mesmos são publicos e disponibilizados para download livremente, mas sim por conta do custo de execução via API baseada em `$/MI token`.

Pipeline

O fluxo de execução da solução segue o seguinte fluxo:



Chunking

Para a realização do Chunking do corpus foi utilizada a biblioteca `langchain_text_splitters` que importa a classe `RecursiveCharacterTextSplitter`, utilizando os separadores

- `separators=["\n\n", "\n", ". ", " ", ""]`

e um valor de `chunksize` de 500

## Método de recuperação

Todos os documentos armazenados no diretório `data` após o procedimento de embedding, foram armazenados em uma collection utilizando o banco de dados vetorizado `chromadb`. A collection foi salva no diretório `./chromedb`

## Pontos de melhorias, controles propostos, limitações e instrução de execução

Todas estas informações estão descritas no arquivo `README.md`